# **APLIKASI KOMPUTER** dan **BASIS DATA**

PERTEMUAN 5 dan 6: DATABASE TABLE RELATIONSHIP

PRAKTISI MENGAJAR 2023
ANDRI NUR ARIEF (TC 08) - MUHAMMAD ADITYA RIDHARRAHMAN (TC 10)

# Pertemuan 5 – Database Table Relationship

1. Foreign Key

2. Join Table
    a. Inner Join
    b. Left Join
    c. Right Join
    d. Full Outer Join

3. Python Introduction
    a. Setup (Python, VSCode and Git) check
    b. Repository preparation
    c. Python Intro
    d. Tuple, Array and Dictionary data type

4. PostgreSQL and Python Connection
    a. What is PIP
    b. Psycopg2 installation
    c. Query data using pyscopg2

5. Git Operations
    a. Git add
    b. Git commit
    c. Git push

# Precondition

Buatlah table dengan criteria sebagai berikut

nama table: buku_perpustakaan_pusat
Column:
- book_id – serial primary key
- book_name - varchar(200)
- book_category – varchar(100)

nama table: peminjaman_buku
Column:
- borrow_id – serial primary key
- student_nim - int
- book_id – int
- borrow_time – date
- return_time - date

# Precondition

```
create table buku_perpustakaan_pusat (
book_id serial primary key,
book_name varchar(200),
book_category varchar(100)
)
```

```
create table peminjaman_buku (
borrow_id serial primary key,
student_nim int4,
book_id int4,
borrow_time date,
return_time date
);
```

# Alter to Serial

```
CREATE SEQUENCE book_id_sequence

ALTER TABLE mahasiswa
ALTER COLUMN book_id SET DEFAULT nextval(book_id_sequence)

ALTER TABLE mahasiswa
ALTER COLUMN book_id SET NOT NULL;
```

# Foreign Key

A foreign key (often abbreviated as FK) is a relational database concept that establishes a link between two tables in a database by defining a relationship between them. It enforces referential integrity and helps maintain the consistency and validity of data in the database. Here are the key aspects of a foreign key:

**Relationship**
A foreign key defines a relationship between two tables, typically referred to as the "parent" table and the "child" table. The parent table contains the primary key, and the child table contains the foreign key that references the primary key.

**Constraints**
A foreign key creates a constraint that restricts the values that can be inserted into the foreign key column of the child table. These values must exist in the primary key column of the parent table.

**Cascade Actions**
Foreign keys can be configured to perform cascade actions when related records in the parent table are updated or deleted. Common cascade actions include "CASCADE" (update or delete related records), "SET NULL" (set the foreign key to NULL), or "SET DEFAULT" (set the foreign key to its default value).

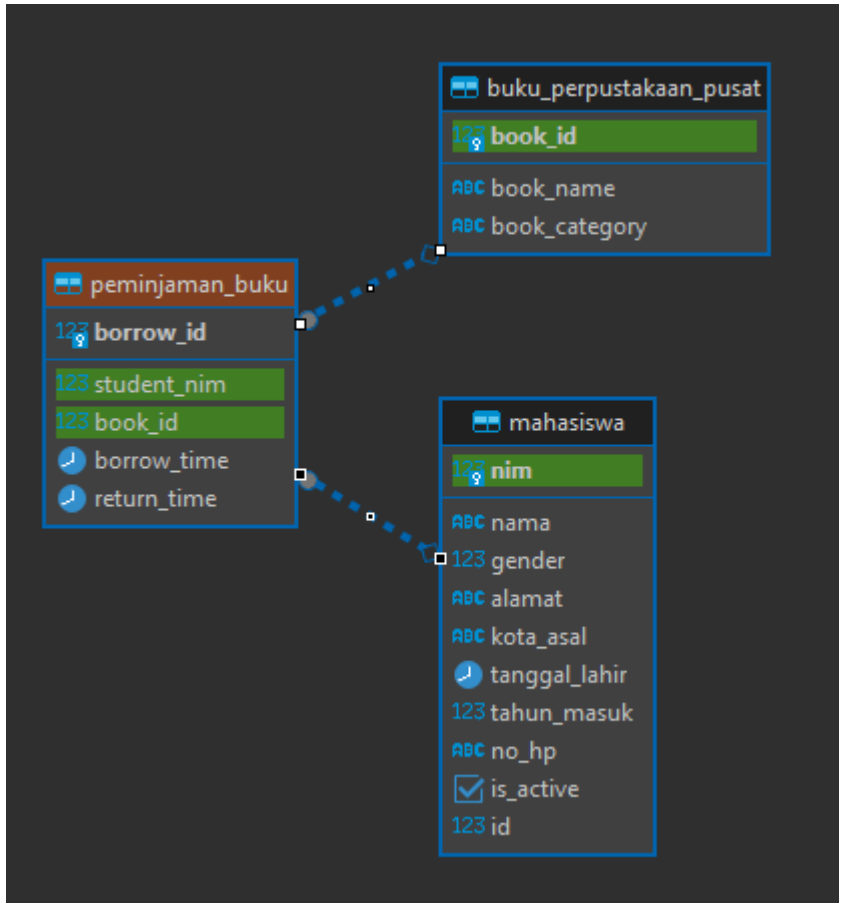
**Improved Querying**
Foreign keys can be used to create joins between tables, making it easier to retrieve and analyze data that spans multiple tables.

# Foreign Key

```
alter table peminjaman_buku
add constraint fk_student_nim
foreign key (student_nim)
references mahasiswa (nim);
```

```
alter table peminjaman_buku
add constraint fk_book_id
foreign key (book_id)
references buku_perpustakaan_pusat (book_id);
```

# Foreign Key


buku_perpustakaan_pusat
book_id
book_name
book_category
peminjaman_buku
borrow_id
student_nim
book_id
borrow_time
return_time
mahasiswa
nim
nama
gender
alamat
kota_asal
tanggal_lahir
tahun_masuk
no_hp
is_active
id

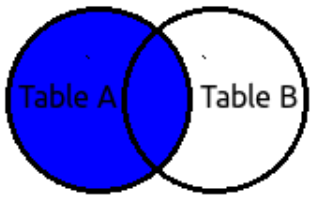
# Exercise Foreign Key

1. Insert data to `buku_perpustakaan_pusat` table [v]
2. Insert data to 'peminjaman_buku' table

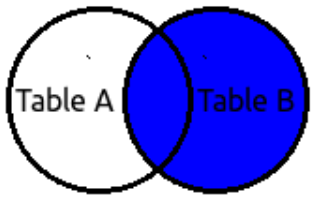
   ```
   INSERT INTO peminjaman_buku (book_id, student_id, borrow_date, return_date)
   values (3, 221331033, 2023-11-08, 2023-11-11)
   ```

3. Delete book data in `buku_perpustakaan_pusat` table where the book is borrowed, what is the result?
4. Delete `mahasiswa` table, what is the result
5. Delete one row in `peminjaman_buku`

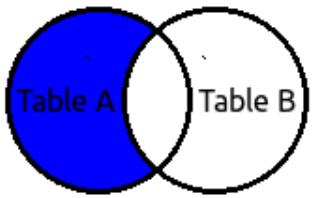
# Join Table



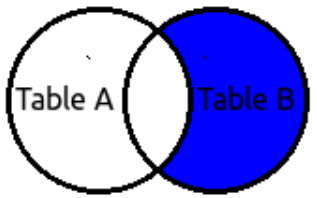

```
SELECT [list] FROM
    [Table A] A
LEFT JOIN
    [Table B] B
ON A.Value = B.Value
```



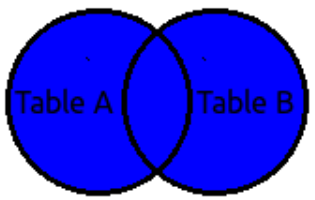

```
SELECT [list] FROM
    [Table A] A
RIGHT JOIN
    [Table B] B
ON A.Value = B.Value
```



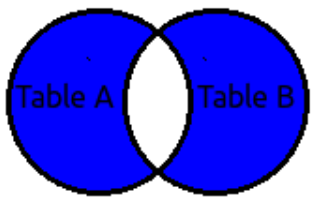

```
SELECT [list] FROM
    [Table A] A
LEFT JOIN
    [Table B] B
ON A.Value = B.Value
WHERE B.Value IS NULL
```



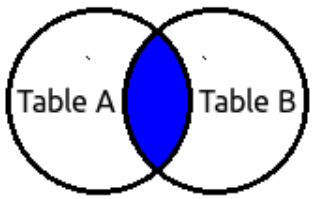

```
SELECT [list] FROM
    [Table A] A
RIGHT JOIN
    [Table B] B
ON A.Value = B.Value
WHERE A.Value IS NULL
```




```
SELECT [list] FROM
    [Table A] A
FULL OUTER JOIN
    [Table B] B
ON A.Value = B.Value
```




```
SELECT [list] FROM
    [Table A] A
FULL OUTER JOIN
    [Table B] B
ON A.Value = B.Value
WHERE A.Value IS NULL
OR B.Value IS NULL
```




```
SELECT [list] FROM
    [Table A] A
INNER JOIN
    [Table B] B
ON A.Value = B.Value
```

# Inner Join

**Inner join**, often referred to simply as a "join," is one of the most common types of joins in relational databases. It is used to combine rows from two or more tables based on a related column between them. **The result of an inner join includes only the rows that have matching values in both tables.**

```
SELECT
    m.nim AS student_nim,
    m.nama AS student_name,
    bp.book_id,
    bp.book_name,
    pb.borrow_time,
    pb.return_time
FROM
   mahasiswa as m
JOIN peminjaman_buku as pb ON m.nim = pb.student_nim
JOIN buku_perpustakaan_pusat as bp ON pb.book_id = bp.book_id;
```

# Full Outer Join

A full outer join combines all rows from both tables, including matching and non-matching rows from both tables.
In cases where there's no match in one of the tables, NULL values are used.

Use cases for FULL OUTER JOIN:

- When you want to retrieve all records from both tables, creating a comprehensive dataset that includes all available information.
- Useful when performing data reconciliation or merging data from different sources.

```
SELECT
     m.nim AS student_nim,
     m.nama AS student_name,
     bp.book_id,
     bp.book_name,
     pb.borrow_time,
     pb.return_time
FROM
   mahasiswa as m
FULL OUTER JOIN peminjaman_buku as pb ON m.nim = pb.student_nim
FULL OUTER JOIN buku_perpustakaan_pusat as bp ON pb.book_id = bp.book_id;
```

# Left Outer Join

In a left outer join, all rows from the left (or first) table and the matching rows from the right (or second) table are included.
If there are no matches in the right table, NULL values are filled in for those columns.

Use cases for LEFT OUTER JOIN:
•When you want to retrieve all records from one table and only matching records from the other.
•For cases where you want to see additional details from the left table even if there are no matches in the right table.

```
SELECT
     m.nim AS student_nim,
     m.nama AS student_name,
     bp.book_id,
     bp.book_name,
     pb.borrow_time,
     pb.return_time
FROM
   mahasiswa as m
LEFT JOIN peminjaman_buku as pb ON m.nim = pb.student_nim
LEFT JOIN buku_perpustakaan_pusat as bp ON pb.book_id = bp.book_id;
```

# Right Outer Join

A right outer join is essentially the opposite of a left outer join. All rows from the right table and the matching rows from the left table are included. If there are no matches in the left table, NULL values are used for those columns.

```
SELECT
    m.nim AS student_nim,
    m.nama AS student_name,
    bp.book_id,
    bp.book_name,
    pb.borrow_time,
    pb.return_time
FROM
   mahasiswa as m
RIGHT JOIN peminjaman_buku as pb ON m.nim = pb.student_nim
RIGHT JOIN buku_perpustakaan_pusat as bp ON pb.book_id = bp.book_id;
```

# Tugas

- Buatlah kelompok dengan 1 kelompok terdiri dari 4 orang
- Buatlah rancangan database dan relasinya dari salah satu opsi dibawah untuk setiap kelompok
  - E-commerce database
  - Hospital database
  - School database
  - Government database (2 kelompok)
  - Police database (e-tilang)
  - Company database

# Python introduction – Setup Check

Check setup

1. Open terminal or command prompt
2. Type `**pyton --version**` or `**py --version**`

```
C:\Users\adity>python --version
Python 3.12.0

C:\Users\adity>py --version
Python 3.12.0
```

3. Type `**git --version**`

```
C:\Users\adity>git --version
git version 2.33.0.windows.2
```

# Python introduction – Repository Preparation

1. Set username on local Git

```
$ git config --global user.name “adityaridha“
$ git config --global user.email adityaridha@example.com
```

2. Create a Repo on GitHub with name ‘python-basis-data’
3. Clone The Repo

# Why Python for Data Engineering and Analysist?

**Rich Ecosystem**
Python has a rich ecosystem of libraries and frameworks that are specifically designed for data-related tasks. Libraries like NumPy, pandas, Matplotlib, Seaborn, and scikit-learn provide powerful tools for data manipulation, analysis, and machine learning.

**Python is a general-purpose, programming language.**
Because of its ease of use and various libraries for accessing databases and storage technologies, it has become a popular tool to execute ETL jobs. Many teams use Python for Data Engineering rather than an ETL tool because it is more versatile and powerful for these activities.

**Learning Curve**
In addition to having support communities, they are both functional and object-oriented languages. Because of its high-level functional characteristics, Java is a bit more complex than Python to master. For simple intuitive logic, Python is preferable, whereas Java is better used in complex workflows. Concise syntax and good standard libraries are provided by Python.

# Python Libraries Related to Data

1. **NumPy (Numerical Python)**
   is a fundamental library for numerical and scientific computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

2. **Pandas**
   is a data manipulation library that offers easy-to-use data structures, such as data frames and series, for data analysis and cleaning. It simplifies tasks like reading and writing data from various file formats, data exploration, and data transformation.

3. **Matplotlib**
   is a popular data visualization library. It provides a wide range of tools for creating static, animated, or interactive plots and graphs, allowing users to visualize data in different formats

4. **OpenCV**
   OpenCV (Open Source Computer Vision Library) is a library for computer vision and image processing tasks. It provides tools for image and video analysis, object detection, facial recognition, and more.

5. **PyTorch**
   These are popular deep learning frameworks for building and training neural networks. They are widely used in various machine learning and deep learning applications, including image recognition, natural language processing, and more.

# Tuple

A tuple is an ordered, immutable collection of elements enclosed in parentheses ().Tuples are typically used when you have a collection of items that should not be changed once created. Elements in a tuple can be of different types, and you can access them using indexing.

**example:**

```
my_tuple = (1, "apple", 3.14)
```

**exercise:**

1. Print first data of the tuple
2. Try update the value

# List

A list is a built-in data structure in Python that can hold an ordered collection of items. Lists are dynamic, which means you can change their size and content. You can store items of varying data types in a list.

Lists are part of Python's standard library, and they are versatile for a wide range of general-purpose data storage and manipulation tasks.

**example:**

```
my_list = contoh_list = [1, 2, 3, ’sebuah string’]
```

**exercise:**

1. Print last data of the list
2. Try update the value

# Dictionary

A dictionary is an unordered collection of key-value pairs enclosed in curly braces {}.
Each key in a dictionary is unique, and it is associated with a value.
Dictionaries are used when you want to store and retrieve data using a unique identifier (the key).

**example:**

```
my_dict = {
    "name": "John",
    "age": 30,
    "city": "New York"
    }
```

**exercise:**

1. Print data ‘name’
2. Update data ‘age’

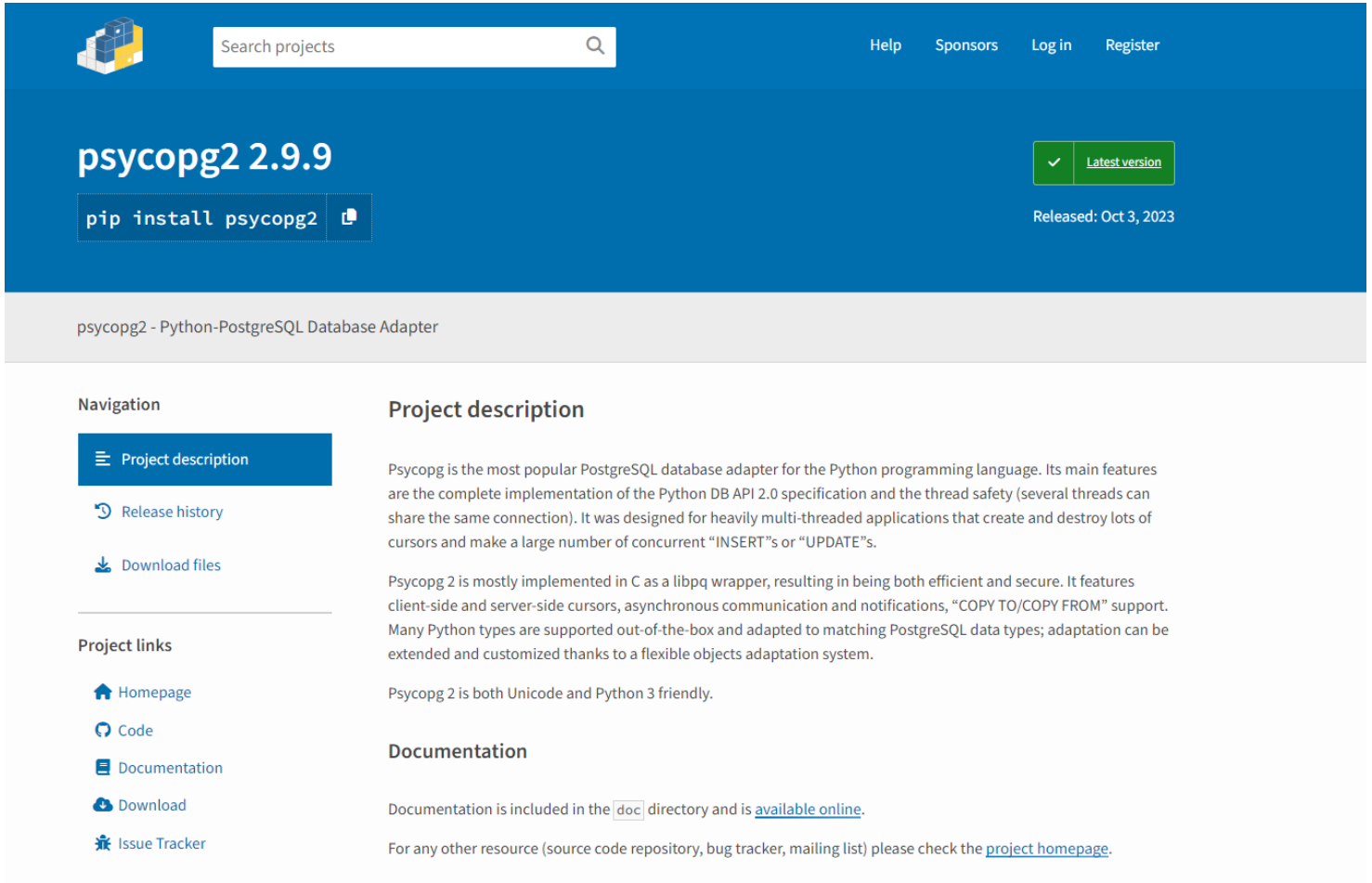
# PIP

PIP, which stands for "Pip Installs Packages," is the default package manager for Python. It is used for installing, managing, and maintaining Python packages and libraries. PIP simplifies the process of distributing and installing Python software, making it easier for developers to work with third-party libraries and packages.

Here are some common PIP commands and functions:

1. **Installing Packages**
   pip install package_name
2. **Upgrading Packages**
   pip install --upgrade package_name
3. **Uninstalling Packages**
   pip uninstall package_name
4. **Listing Installed Packages**
   pip list
5. **Freezing Requirements**
   pip freeze > requirements.txt
6. **Installing Packages from `requirements.txt`**
   pip install -r requirements.txt

# Psycopg2


Search projects
Help
Sponsors
Log in
Register
psycopg2 2.9.9
pip install psycopg2
Latest version
Released: Oct 3, 2023
psycopg2 - Python-PostgreSQL Database Adapter
Navigation
Project description
Release history
Download files
Project links
Homepage
Code
Documentation
Download
Issue Tracker
Project description
Psycopg is the most popular PostgreSQL database adapter for the Python programming language. Its main features are the complete implementation of the Python DB API 2.0 specification and the thread safety (several threads can share the same connection). It was designed for heavily multi-threaded applications that create and destroy lots of cursors and make a large number of concurrent “INSERT”s or “UPDATE”s.
Psycopg 2 is mostly implemented in C as a libpq wrapper, resulting in being both efficient and secure. It features client-side and server-side cursors, asynchronous communication and notifications, “COPY TO/COPY FROM” support. Many Python types are supported out-of-the-box and adapted to matching PostgreSQL data types; adaptation can be extended and customized thanks to a flexible objects adaptation system.
Psycopg 2 is both Unicode and Python 3 friendly.
Documentation
Documentation is included in the doc directory and is available online.
For any other resource (source code repository, bug tracker, mailing list) please check the project homepage.

# Query From Psycopg2

https://github.com/adityaridha/python-basis-data

# Insert Data From Psycopg2

# GIT PUSH

GIT STATUS
GIT ADD . Atau GIT ADD <filename>
GIT COMMIT –M “commit message teman teman”
GIT PUSH